**VISÃO DO PRODUTO**

Plataforma de ride-sharing (TVDE).

Objetivo:

- Validar modelo de negócio
- Backend robusto
- Fluxo financeiro correto
- Preparação futura para Stripe Connect (split motorista)

Estado atual:

- Backend FastAPI
- PostgreSQL
- Stripe manual capture
- Web Test Console (React/Vite)
- ENV=dev
- create_all em vez de migrations

---

## 2️⃣ ARQUITETURA GERAL

Stack:

- FastAPI
- SQLAlchemy 2.x
- PostgreSQL
- Stripe API
- JWT + OTP
- Pydantic Settings
- Docker (Postgres)

Sem Alembic.
Sem Stripe Connect.
Sem frontend produção ainda.

---

## 3️⃣ STATE MACHINE — TRIP

requested
→ assigned
→ accepted
→ arriving

→ ongoing
→ completed

Regras importantes:

- complete só permitido se status == ongoing
- double complete retorna 409
- double accept bloqueado
- cancel permitido conforme role

---

## 🔷 MODELO FINANCEIRO ATUAL

**ACCEPT_TRIP**

- create_authorization_payment_intent(confirm=False)
- capture_method="manual"
- status Stripe: requires_confirmation
- Payment interno criado com:
  - status=processing
  - total_amount=0.50 (placeholder inicial)
- idempotência: UniqueConstraint(trip_id)

---

**COMPLETE_TRIP**

Fluxo:

1. Validar trip.status == ongoing
2. Calcular final_price via pricing engine
3. commission_amount = final_price × driver.commission_percent
4. driver_payout = final_price - commission_amount
5. update_payment_intent_amount
6. confirm_payment_intent
7. capture_payment_intent
8. Só após capture OK:
   - trip.status = completed
   - trip.completed_at preenchido
   - db.commit()

Se falha:

- db.rollback()
- HTTP 502
- trip permanece ongoing

Webhook atualiza payment.status para succeeded.

---

### 5️⃣ PRICING ENGINE

app/core/pricing.py

Constantes:

- BASE_FARE
- PER_KM
- PER_MIN

Funções:

- calculate_price(distance_km, duration_min)
- calculate_driver_payout(total)

Arredondamento com helper _money().

---

### 6️⃣ MODELO PAYMENT

Campos importantes:

- total_amount
- commission_amount
- driver_amount (legado)
- driver_payout (final)
- stripe_payment_intent_id
- status (processing / succeeded / failed)

UniqueConstraint(trip_id)

---

### 7️⃣ STRIPE

Manual capture.

Fluxo correto atual:

ACCEPT:
requires_confirmation

COMPLETE:
update → confirm → capture

Webhook:

- payment_intent.succeeded
- charge.succeeded
- idempotente

Sem 3DS.
Sem Connect.
Sem automatic capture.

---

## 8️⃣ TESTES JÁ VALIDADOS

✓ Fluxo feliz
✓ Comissão dinâmica (22%)
✓ Atomicidade sob falha
✓ Recuperação após falha
✓ Webhook idempotente
✓ Double complete bloqueado

---

## 9️⃣ RISCOS CONHECIDOS

- PaymentIntent órfão se commit falhar após create
- Falta de retries Stripe
- No migrations
- authorization_expires_at não usado
- Produção exigirá frontend confirmar PaymentIntent

---

## 🔟 PRÓXIMA FASE

Fase 2: Web App responsiva real
Mas só após hardening financeiro completo.